

Corrections d'exercices

Catégorie : TD/TP
Matière : Algorithmique et Programmation I
Libellé : TD/TP sur le chapitre : STRUCTURES DE CONTRÔLE / STRUCTURE CONDITIONNELLE - A. BASSE & I. GAYE - 18/07/23

Indications : Faire l'analyse avant chaque exercice

- Les données d'entrées
- Les données de sorties
- Les données intermédiaires
- La méthode (principe de résolution)

Exercice 1 :

Ecrire un algorithme (algorigramme ou pseudo-code) qui affiche la parité d'un nombre lu en utilisant :

1. une structure au sens strict
2. une structure alternative

Analyse :
* Données d'entrée : un nombre entier
* Données de sortie : la parité du nombre (pair ou impair)
* Données intermédiaires : aucune
* Méthode :
   1. Lire le nombre entier.
   2. Si le nombre est divisible par 2 (reste de la division par 2 est égal à 0) alors afficher "Le nombre est pair".
   3. Sinon afficher "Le nombre est impair".

1. Structure au sens strict :

```
Algorithme AfficherPariteSensStrict;
    Variable nombre_saisi : entier;
    Début
        Ecrire("Saisir un nombre entier : ");
        Lire (nombre_saisi);

        Si nombre_saisi % 2 = 0 Alors
            Ecrire("Le nombre est pair");
        FinSi

        Si nombre_saisi % 2 <> 0 Alors
            Ecrire("Le nombre est impair");
        FinSi
    Fin.
```

2. Structure alternative :

```
Algorithme AfficherPariteAlternative;
    Variable nombre_saisi : entier;
    Début
        Ecrire("Saisir un nombre entier : ");
        Lire (nombre_saisi);

        Si nombre_saisi % 2 = 0 Alors
            Ecrire("Le nombre est pair");
        Sinon
            Ecrire("Le nombre est impair");
        FinSi
    Fin.
```

Exercice 2 :

Ecrire un algorithme (algorigramme ou pseudo-code) qui affiche si un nombre lu est positif, négatif ou nul en utilisant :

1. une structure au sens strict (Si ... Alors)
2. une structure alternative (Si ... Alors ... Sinon)

Analyse :
* Données d'entrée : un nombre
* Données de sortie : un message indiquant si le nombre est positif, négatif ou nul
* Données intermédiaires : aucune
* Méthode :
  1. Lire le nombre.
  2. Si le nombre est positif, afficher "Le nombre est positif".
  3. Sinon, si le nombre est négatif, afficher "Le nombre est négatif".
  4. Sinon, afficher "Le nombre est nul".

1. Structure au sens strict :

```
Algorithme AfficherSigneSenseStrict;
    Variable nb : réel;
    Début
        Ecrire("Entrez un nombre : ");
        Lire(nb);

        Si nb > 0 Alors
            Ecrire("Le nombre est positif");
        FinSi

        Si nb < 0 Alors
            Ecrire("Le nombre est négatif");
        FinSi

        Si nb = 0 Alors
            Ecrire("Le nombre est nul");
        FinSi
    Fin.
```

2. Structure alternative :

```
Algorithme AfficherSigneAlternative;
    Variable nb : réel;
    Début
        Ecrire("Entrez un nombre : ");
        Lire(nb);

        Si nb > 0 Alors
            Ecrire("Le nombre est positif");
        Sinon
            Si nb < 0 Alors
                Ecrire("Le nombre est négatif");
            Sinon
                Ecrire("Le nombre est nul");
            FinSi
        FinSi
    Fin.
```

Exercice 3 :

Proposer un algorithme (algorigramme ou pseudo-code) permettant d'afficher le double d'un nombre donné si celui-ci est inférieur à un nombre seuil en utilisant :

1. une structure au sens strict (Si ... Alors)
2. une structure alternative (Si ... Alors ... Sinon)

Analyse :
* Données d'entrée : un nombre seuil et un nombre quelconque
* Données de sortie : le double du nombre si celui-ci est inférieur à un nombre seuil
* Données intermédiaires : aucune
* Méthode :
  1. Lire le nombre quelconque.
  2. Lire le nombre seuil.
  3. Si le nombre est inférieur au nombre seuil, afficher le double du nombre.

1. Structure au sens strict :

```
Algorithme AfficherDoubleSensStrict;
    Variable nb, seuil : réel;
    Début
        Ecrire("Entrez un nombre : ");
        Lire(nb);
        Ecrire("Entrez un nombre seuil : ");
        Lire(seuil);

        Si nb < seuil Alors
            Ecrire("Le double du nombre est ", nb * 2);
        FinSi
    Fin.
```

2. Structure alternative :

```
Algorithme AfficherDoubleAlternative;
    Variable nb, seuil : réel;
    Début
        Ecrire("Entrez un nombre : ");
        Lire(nb);
        Ecrire("Entrez un nombre seuil : ");
        Lire(seuil);

        Si nb < seuil Alors
            Ecrire("Le double du nombre est ", nb * 2);
        Sinon
            Ecrire("Le nombre saisi est supérieur au nombre seuil");
        FinSi
    Fin.
```

Exercice 4 :

Proposer un algorithme (algorigramme ou pseudo-code) qui affiche le plus grand entre deux entiers lus en utilisant :

1. une structure au sens strict (Si ... Alors)
2. une structure alternative (Si ... Alors ... Sinon)

Analyse :
* Données d'entrée : deux entiers
* Données de sortie : le plus grand des deux entiers
* Données intermédiaires : aucune
* Méthode :
  1. Lire le premier entier.
  2. Lire le deuxième entier.
  3. Initialiser le plus grand entier à la valeur du premier entier.
  4. Si le deuxième entier est supérieur au premier entier, initialiser le plus grand entier à la valeur du deuxième entier.
  5. Afficher le plus grand entier.

1. Structure au sens strict :

```
Algorithme AfficherPlusGrandSensStrict;
    Variables nb1, nb2 , plusGrand : entier;
    Début
        Ecrire("Entrez un premier nombre : ");
        Lire(nb1);
        Ecrire("Entrez un deuxième nombre : ");
        Lire(nb2);

        plusGrand <- nb1;

        Si plusGrand < nb2 Alors
            plusGrand <- nb2;
        FinSi

        Ecrire("Le plus grand nombre est ", plusGrand);
    Fin.
```

2. Structure alternative :

```
Algorithme AfficherPlusGrandAlternative;
    Variables nb1, nb2 , plusGrand : entier;
    Début
        Ecrire("Entrez un premier nombre : ");
        Lire(nb1);
        Ecrire("Entrez un deuxième nombre : ");
        Lire(nb2);

        plusGrand <- nb1;

        Si plusGrand < nb2 Alors
            Debut
                plusGrand <- nb2;
                Ecrire("Le plus grand nombre est ", plusGrand);
            Fin
        Sinon
            Ecrire("Le plus grand nombre est ", plusGrand);
        FinSi
    Fin.
```

Exercice 5 :

Proposer un algorithme (algorigramme ou pseudo-code) qui affiche le plus grand entre trois entiers lus en utilisant :

1. une structure au sens strict (Si ... Alors)
2. une structure alternative (Si ... Alors ... Sinon)

Analyse :
* Données d'entrée : trois entiers
* Données de sortie : le plus grand des trois entiers
* Données intermédiaires : aucune
* Méthode :
  1. Lire le premier entier.
  2. Lire le deuxième entier.
  3. Lire le troisième entier.
  4. Initialiser le plus grand entier à la valeur du premier entier.
  5. Si le deuxième entier est supérieur au plus grand entier, initialiser le plus grand entier à la valeur du deuxième entier.
  6. Si le troisième entier est supérieur au plus grand entier, initialiser le plus grand entier à la valeur du troisième entier.
  7. Afficher le plus grand entier.

1. Structure au sens strict :

```
Algorithme AfficherPlusGrandSensStrict;
    Variables nb1, nb2, nb3, plusGrand : entier;
    Début
        Ecrire("Entrez un premier nombre : ");
        Lire(nb1);
        Ecrire("Entrez un deuxième nombre : ");
        Lire(nb2);
        Ecrire("Entrez un troisième nombre : ");
        Lire(nb3);

        plusGrand <- nb1;

        Si plusGrand < nb2 Alors
            plusGrand <- nb2;
        FinSi

        Si plusGrand < nb3 Alors
            plusGrand <- nb3;
        FinSi

        Ecrire("Le plus grand nombre est ", plusGrand);
    Fin.
```

2. Structure alternative :

```
Algorithme AfficherPlusGrandAlternative;
    Variables nb1, nb2, nb3, plusGrand : entier;
    Début
        Ecrire("Entrez un premier nombre : ");
        Lire(nb1);
        Ecrire("Entrez un deuxième nombre : ");
        Lire(nb2);
        Ecrire("Entrez un troisième nombre : ");
        Lire(nb3);

        plusGrand <- nb1;

        Si plusGrand < nb2 Alors
            Debut
                plusGrand <- nb2;
                Si plusGrand < nb3 Alors
                    Debut
                        plusGrand <- nb3;
                        Ecrire("Le plus grand nombre est ", plusGrand);
                    Fin
                Sinon
                    Ecrire("Le plus grand nombre est ", plusGrand);
                FinSi
            Fin
        Sinon
            Si plusGrand < nb3 Alors
                Debut
                    plusGrand <- nb3;
                    Ecrire("Le plus grand nombre est ", plusGrand);
                Fin
            Sinon
                Ecrire("Le plus grand nombre est ", plusGrand);
            FinSi
        FinSi
    Fin.
```

Exercice 6 :

Proposer un algorithme (algorigramme ou pseudo-code) qui lit trois valeurs entières (A, B et C) au clavier et les affiche dans l'ordre croissant en utilisant :

1. une structure au sens strict (Si ... Alors)
2. une structure alternative (Si ... Alors ... Sinon)

Analyse :
* Données d'entrée : trois entiers A, B et C
* Données de sortie : les trois entiers dans l'ordre croissant
* Données intermédiaires : variable temporaire
* Méthode :
  1. Lire le premier entier A.
  2. Lire le deuxième entier B.
  3. Lire le troisième entier C.
  4. Si A est supérieur à B, alors echanger les valeurs de A et B.
  5. Si B est supérieur à C, alors echanger les valeurs de B et C.
  6. Si A est supérieur à B, alors echanger les valeurs de A et B.
  7. Afficher les trois entiers dans l'ordre croissant.

1. Structure au sens strict :

```
Algorithme AfficherOrdreCroissantSensStrict;
    Variables A, B, C, temp : entier;
    Début
        Ecrire("Entrez un premier nombre : ");
        Lire(A);
        Ecrire("Entrez un deuxième nombre : ");
        Lire(B);
        Ecrire("Entrez un troisième nombre : ");
        Lire(C);

        Si A > B Alors
            temp <- A;
            A <- B;
            B <- temp;
        FinSi

        Si B > C Alors
            temp <- B;
            B <- C;
            C <- temp;
        FinSi

        Si A > B Alors
            temp <- A;
            A <- B;
            B <- temp;
        FinSi

        Ecrire("Les nombres dans l'ordre croissant sont ", A, ", ", B, ", ", C);
    Fin.
```

2. Structure alternative :

```
Algorithme AfficherOrdreCroissantAlternative;
    Variables A, B, C, temp : entier;
    Début
        Ecrire("Entrez un premier nombre : ");
        Lire(A);
        Ecrire("Entrez un deuxième nombre : ");
        Lire(B);
        Ecrire("Entrez un troisième nombre : ");
        Lire(C);

        Si A > B Alors
            Debut
                temp <- A;
                A <- B;
                B <- temp;
                Si B > C Alors
                    Debut
                        temp <- B;
                        B <- C;
                        C <- temp;
                        Si A > B Alors
                            Debut
                                temp <- A;
                                A <- B;
                                B <- temp;
                            Fin
                        FinSi
                    Fin
                FinSi
            Fin
        Sinon
            Si B > C Alors
                Debut
                    temp <- B;
                    B <- C;
                    C <- temp;
                    Si A > B Alors
                        Debut
                            temp <- A;
                            A <- B;
                            B <- temp;
                        Fin
                    FinSi
                Fin
            FinSi
        FinSi

        Ecrire("Les nombres dans l'ordre croissant sont ", A, ", ", B, ", ", C);
    Fin.
```

Exercice 7 :

Proposer un programme Pascal qui dispose de trois valeurs entières et qui affiche d'ordre dans lequel ils ont été saisis : soit "Ordre croissant", "Ordre décroissant" ou "Désordonné".

Analyse :
* Données d'entrée : trois entiers
* Données de sortie : "Ordre croissant", "Ordre décroissant" ou "Désordonné"
* Données intermédiaires : aucune
* Méthode :
  1. Lire le premier entier.
  2. Lire le deuxième entier.
  3. Lire le troisième entier.
  4. Si le premier entier est inférieur ou égal au deuxième entier
     et que le deuxième entier est inférieur au troisième entier,
     ou si le premier entier est inférieur au deuxième entier
     et que le deuxième entier est inférieur ou égal au troisième entier,
     afficher "Ordre croissant".
  5. Sinon, si le premier entier est supérieur ou égal au deuxième entier
     et que le deuxième entier est supérieur au troisième entier,
     ou si le premier entier est supérieur au deuxième entier
     et que le deuxième entier est supérieur ou égal au troisième entier,
     afficher "Ordre décroissant".
  6. Sinon, afficher "Désordonné".

```
Program AfficherOrdre;
    Var nb1, nb2, nb3 : integer;
    Begin
        writeln('Entrez un premier nombre : ');
        readln(nb1);
        writeln('Entrez un deuxième nombre : ');
        readln(nb2);
        writeln('Entrez un troisième nombre : ');
        readln(nb3);

        If (nb1 <= nb2) AND (nb2 < nb3)
        OR (nb1 < nb2) AND (nb2 <= nb3) Then
            writeln('Ordre croissant')
        Else
            If (nb1 >= nb2) AND (nb2 > nb3)
            OR (nb1 > nb2) AND (nb2 >= nb3) Then
                writeln('Ordre décroissant')
            Else
                writeln('Désordonné');
    End.
```

Exercice 8 :

Ecrire un algorithme permettant de résoudre dans R :

1. Une équation du premier degré ($aX + b = 0$)
2. Une équation du second degré ($aX^2 + bX + c = 0$)

Analyse :

* Données d'entrée : les coefficients a, b et c
* Données de sortie : les solutions de l'équation
* Données intermédiaires : le discriminant
* Méthode :
    1. Lire le coefficient a.
    2. Lire le coefficient b.
    3. Lire le coefficient c.
    4. Si le coefficient a est égal à 0, afficher "L'équation n'est pas du premier degré".
    5. Sinon, si le coefficient a est différent de 0, calculer le discriminant.
    6. Si le discriminant est supérieur à 0, afficher les deux solutions de l'équation.
    7. Sinon, si le discriminant est égal à 0, afficher la solution de l'équation.
    8. Sinon, afficher "L'équation n'a pas de solution réelle".

```
Algorithme ResoudreEquation;
    Variable a, b, c, discriminant, x1, x2, x0 : réel;
    Début
        Ecrire("Entrez le coefficient a : ");
        Lire(a);
        Ecrire("Entrez le coefficient b : ");
        Lire(b);
        Ecrire("Entrez le coefficient c : ");
        Lire(c);

        Si a = 0 Alors
            Ecrire("L'équation n'est pas du premier degré");
        Sinon
            Debut
                discriminant <- (b * b) - (4 * a * c);
                Si discriminant > 0 Alors
                    Début
                        x1 <- (-b - racineCarree(discriminant)) / (2 * a);
                        x2 <- (-b + racineCarree(discriminant)) / (2 * a);
                        Ecrire("Les deux solutions de l'équation sont "
                      Ecrire(x1, " et ", x2);
                    Fin
                Sinon
                    Si discriminant = 0 Alors
                        Début
                            x0 <- -b / (2 * a);
                            Ecrire("La solution de l'équation est ", x0);
                        Fin
                    Sinon
                        Ecrire("L'équation n'a pas de solution réelle");
                    FinSi
                FinSi
            Fin
        FinSi
    Fin.
```

Exercice 9 :

Ecrire un algorithme (algorigramme ou pseudo-code) permettant de simuler une calculatrice à quatre opérations (+, -, *, /) en utilisant :

1. une structure au sens strict (Si ... Alors)
2. une structure alternative (TP en Pascal)
3. une structure Selon ... Faire (TP : en Pascal et TD)

Analyse :
* Données d'entrée : deux nombres, un opérateur
* Données de sortie : le résultat de l'opération
* Données intermédiaires : aucune
* Méthode :
  1. Lire le premier nombre.
  2. Lire l'opérateur.
  3. Lire le deuxième nombre.
  4. Si l'opérateur est égal à "+", afficher le résultat de l'addition.
  5. Sinon, si l'opérateur est égal à "-", afficher le résultat de la soustraction.
  6. Sinon, si l'opérateur est égal à "*", afficher le résultat de la multiplication.
  7. Sinon, si l'opérateur est égal à "/", vérifier que le deuxième nombre est différent de 0. Si le deuxième nombre est différent de 0, afficher le résultat de la division.
  8. Sinon, afficher "L'opérateur saisi est incorrect".

1. Structure au sens strict :

```
Algorithme CalculatriceSensStrict;
    Variable nb1, nb2, resultat : réel;
    Début
        Ecrire("Entrez un premier nombre : ");
        Lire(nb1);
        Ecrire("Entrez un opérateur : ");
        Lire(operateur);
        Ecrire("Entrez un deuxième nombre : ");
        Lire(nb2);

        Si operateur = "+" Alors
            Début
                resultat <- nb1 + nb2;
                Ecrire("Le résultat de l'addition est ", resultat);
            Fin
        FinSi

        Si operateur = "-" Alors
            Début
                resultat <- nb1 - nb2;
                Ecrire("Le résultat de la soustraction est ", resultat);
            Fin
        FinSi

        Si operateur = "*" Alors
            Début
                resultat <- nb1 * nb2;
                Ecrire("Le résultat de la multiplication est ", resultat);
            Fin
        FinSi

        Si operateur = "/" Alors
             Début
                Si nb2 <> 0 alors
                    Début
                        resultat <- nb1 / nb2;
                        Ecrire("Le résultat de la division est ", resultat);
                     Fin
                FinSi
                Si nb2 = 0 alors
                    Ecrire("Le deuxième nombre ne peut pas être égal à 0");
                FinSi
             Fin
        FinSi

        Si operateur <> "+" ET operateur <> "-"
         ET operateur <> "*" ET operateur <> "/" Alors
            Ecrire("L'opérateur saisi est incorrect");
        FinSi
    Fin.
```

2. Structure alternative :

```
Program CalculatriceAlternative;
  Var nb1, nb2, resultat : real;
      operateur : char;
  Begin
      writeln('Entrez un premier nombre : ');
      readln(nb1);
      writeln('Entrez un opérateur : ');
      readln(operateur);
      writeln('Entrez un deuxième nombre : ');
      readln(nb2);

      If operateur = '+' Then
          Begin
              resultat := nb1 + nb2;
              write('Le résultat de l''addition est');
              write(' ', resultat:0:2);
          End
      Else
          If operateur = '-' Then
              Begin
                  resultat := nb1 - nb2;
                  write('Le résultat de la soustraction est');
                  write(' ', resultat:0:2);
              End
          Else
              If operateur = '*' Then
                  Begin
                      resultat := nb1 * nb2;
                      write('Le résultat de la multiplication est');
                      write(' ', resultat:0:2);
                  End
              Else
                  If operateur = '/' Then
                      If nb2 <> 0 Then
                          Begin
                              resultat := nb1 / nb2;
                              write('Le résultat de la division est');
                              write(' ', resultat:0:2);
                          End
                      Else
                          writeln('Le deuxième nombre ne peut pas être égal à 0')
                  Else
                        writeln('L''opérateur saisi est incorrect');
    End.
```

3. Structure Selon ... Faire :

```
Program CalculatriceSelonFaire;
    Var nb1, nb2, resultat : real;
        operateur : char;
    Begin
        writeln('Entrez un premier nombre : ');
        readln(nb1);
        writeln('Entrez un opérateur : ');
        readln(operateur);
        writeln('Entrez un deuxième nombre : ');
        readln(nb2);

        Case operateur of
            '+' : Begin
                    resultat := nb1 + nb2;
                    write('Le résultat de l''addition est ', resultat:0:2);
                  End;
            '-' : Begin
                    resultat := nb1 - nb2;
                    write('Le résultat de la soustraction est ', resultat:0:2);
                  End;
            '*' : Begin
                    resultat := nb1 * nb2;
                    write('Le résultat de la multiplication est ', resultat:0:2);
                  End;
            '/' : Begin
                    If nb2 <> 0 Then
                        Begin
                            resultat := nb1 / nb2;
                            write('Le résultat de la division est ', resultat:0:2);
                        End
                    Else
                        writeln('Le deuxième nombre ne peut pas être égal à 0');
                  End
            Else
                writeln('L''opérateur saisi est incorrect');
        End;
    End.
```

Exercice 10 :

Une année est bissextile (contient 366 jours) si elle est multiple de 4, sauf les années de début de siècle (qui se terminent par 00) qui ne sont bissextiles que si elles sont divisibles par 400.
Exemples:
- 1980 et 1996 sont bissextiles car elles sont divisibles par 4
- 2000 est une année bissextile car elle est divisible par 400
- 2100 3000 ne sont pas bissextiles car elles ne sont pas divisibles par 400.

Écrire un programme Pascal qui permet de déterminer s'il un entier positif donné correspond à une année bissextile ou non.

Analyse :
* Données d'entrée : un entier positif
* Données de sortie : "L'année est bissextile" ou "L'année n'est pas bissextile"
* Données intermédiaires : aucune
* Méthode :
  1. Lire l'année.
  2. Si l'année est divisible par 100, vérifier si elle est divisible par 400.
     Si oui, afficher "L'année est bissextile". Sinon, afficher "L'année n'est pas bissextile".
  3. Sinon, si l'année est divisible par 4, afficher "L'année est bissextile".
     Sinon, afficher "L'année n'est pas bissextile".

```
Program AnneeBissextile;
    Var annee : integer;
    Begin
        writeln('Entrez une année : ');
        readln(annee);

        If annee mod 100 = 0 Then
            If annee mod 400 = 0 Then
                writeln('L''année est bissextile')
            Else
                writeln('L''année n''est pas bissextile')
        Else
            If annee mod 4 = 0 Then
                writeln('L''année est bissextile')
            Else
                writeln('L''année n''est pas bissextile');
    End.
```

Exercice 11 :

Écrire un algorithme (algorigramme ou pseudo-code) qui dispose du genre (H, F), du nom d'une personne et qui affiche le nom en la précéder de "Monsieur" ou "Madame" suivant le genre.

Analyse :
* Données d'entrée : le genre (H ou F) et le nom d'une personne
* Données de sortie : le nom précédé de "Monsieur" ou "Madame"
* Données intermédiaires : aucune
* Méthode :
  1. Lire le genre.
  2. Lire le nom.
  3. Si le genre est égal à "H", afficher "Monsieur" suivi du nom.
  4. Sinon, si le genre est égal à "F", afficher "Madame" suivi du nom.
  5. Sinon, afficher "Le genre saisi est incorrect".

```
Algorithme AfficherNom;
    Variable genre, nom : chaine;
    Début
        Ecrire("Entrez le genre (H ou F) : ");
        Lire(genre);
        Ecrire("Entrez le nom : ");
        Lire(nom);

        Si genre = "H" Alors
            Ecrire("Monsieur ", nom);
        Sinon
            Si genre = "F" Alors
                Ecrire("Madame ", nom);
            Sinon
                Ecrire("Le genre saisi est incorrect");
            FinSi
        FinSi
    Fin.
```

Exercice 12 :

Ecrire un algorithme (algorigramme ou pseudo-code) qui dispose du genre (H, F), du nom d'une personne, de son état matrimonial (0 célibataire, 1 marié) et qui réalise un affichage du nom en le faisant précéder de "Monsieur", de "Madame" ou de "Mademoiselle" suivant le genre et éventuellement l'état matrimonial.

Analyse :
* Données d'entrée : le genre (H ou F), le nom d'une personne, l'état matrimonial (0 ou 1)
* Données de sortie : le nom précédé de "Monsieur", "Madame" ou "Mademoiselle"
* Données intermédiaires : aucune
* Méthode :
   1. Lire le genre.
   2. Lire le nom.
   3. Lire l'état matrimonial.
   4. Si le genre est égal à "H", afficher "Monsieur" suivi du nom.
   5. Sinon, si le genre est égal à "F", et que l'état matrimonial est égal à 0, afficher "Mademoiselle" suivi du nom.
   6. Sinon, si le genre est égal à "F", et que l'état matrimonial est égal à 1, afficher "Madame" suivi du nom.
   7. Sinon, afficher "Le genre saisi est incorrect".

```
Algorithme AfficherNom;
    Variable genre, nom : chaine;
        etatMatrimonial : entier;
    Début
        Ecrire("Entrez le genre (H ou F) : ");
        Lire(genre);
        Ecrire("Entrez le nom : ");
        Lire(nom);
        Ecrire("Entrez l'état matrimonial (0 célibataire, 1 marié) : ");
        Lire(etatMatrimonial);

        Si genre = "H" Alors
            Ecrire("Monsieur ", nom);
        Sinon
            Si genre = "F" ET etatMatrimonial = 0 Alors
                Ecrire("Mademoiselle ", nom);
            Sinon
                Si genre = "F" ET etatMatrimonial = 1 Alors
                    Ecrire("Madame ", nom);
                Sinon
                    Ecrire("Le genre saisi est incorrect");
                FinSi
            FinSi
        FinSi
    Fin.
```

Exercice 13 :

Un magasin de vente de légumes propose les prix suivants pour la carotte
selon la quantité achetée. Si la quantité achetée
- est inférieure strictement à 10 kg, le prix du kg est de 350 frs;
- est comprise entre 10 et 19kg, le prix du kg est de 325 frs;
- est comprise entre 20 et 30kg, le prix du kg est de 300 frs;
- est supérieure à 30kg, le prix du kg est de 250 frs.

Écrire un programme Pascal qui permet de calculer le montant de la facture
d'un client et de la monnaie à lui rendre.

Analyse :
* Données d'entrée : la quantité de carottes achetée, le montant versé par le client
* Données de sortie : le montant de la facture et la monnaie à rendre
* Données intermédiaires : le prix du kg de carottes
* Méthode :
   1. Lire la quantité de carottes achetée.
   2. Si la quantité de carottes achetée est inférieure ou égale à 0,
      afficher "La quantité saisie est incorrecte".
   3. Si la quantité de carottes achetée est inférieure à 10 kg, le prix du kg est de 350 frs.
   4. Sinon, si la quantité de carottes achetée est comprise entre 10 et 19kg,
      le prix du kg est de 325 frs.
   5. Sinon, si la quantité de carottes achetée est comprise entre 20 et 30kg,
      le prix du kg est de 300 frs.
   6. Sinon, si la quantité de carottes achetée est supérieure à 30kg,
      le prix du kg est de 250 frs.
   7. Calculer le montant de la facture et l'afficher.
   8. Lire le montant versé par le client.
   9. Si le montant versé par le client est supérieur au montant de la facture,
      calculer la monnaie à rendre et l'afficher.
   10. Sinon, afficher "Le montant versé par le client est insuffisant".

```
Program CalculerFacture;
    Var quantiteCarottes, prixCarottes, montantFacture,
        montantVerse, monnaieRendue : real;
    Begin
        writeln('Entrez la quantité de carottes achetée : ');
        readln(quantiteCarottes);

        If quantiteCarottes <= 0 Then
            writeln('La quantité saisie est incorrecte')
        Else
            If quantiteCarottes < 10 Then
                prixCarottes := 350
            Else
                If quantiteCarottes < 20 Then
                    prixCarottes := 325
                Else
                    If quantiteCarottes < 30 Then
                        prixCarottes := 300
                    Else
                        prixCarottes := 250;

        montantFacture := quantiteCarottes * prixCarottes;
        writeln('Le montant de la facture est de ', montantFacture:0:2);

        writeln('Entrez le montant versé par le client : ');
        readln(montantVerse);

        If montantVerse > montantFacture Then
            Begin
                monnaieRendue := montantVerse - montantFacture;
                writeln('La monnaie à rendre est de ', monnaieRendue:0:2);
            End
        Else
            writeln('Le montant versé par le client est insuffisant');
    End.
```

Exercice 14 :

Le guichet du CROUS permet à un étudiant d'acheter des tickets de restaurant
dont les prix sont fixés comme suit : petit déjeuner à 75 frs, déjeuner et dîner à 150 frs.
Proposer un algorithme (algorigramme ou pseudo code) permettant au guichetier :

- de vérifier si le montant donné par l'étudiant permet d'acheter

le nombre de tickets voulus par celui-ci dans chaque catégorie.
Si cette vérification ne se passe pas bien, le guichetier doit informer l'étudiant
du montant manquant pour valider la commande.

- de calculer la monnaie à rendre si la vérification s'est bien passée.

Analyse :
* Données d'entrée : le nombre de tickets de petit déjeuner, le nombre de tickets de déjeuner,
le nombre de tickets de dîner, le montant versé par l'étudiant.
* Données de sortie : le montant manquant pour valider la commande, la monnaie à rendre.
* Données intermédiaires : le montant de la commande.
* Méthode :

1. Lire le nombre de tickets de petit déjeuner.
2. Lire le nombre de tickets de déjeuner.
3. Lire le nombre de tickets de diner.
4. Lire le montant versé par l'étudiant.
5. Calculer le montant de la commande.
6. Si le montant versé par l'étudiant est supérieur au montant de la commande, calculer la monnaie à rendre et l'afficher.
7. Sinon, calculer le montant manquant pour valider la commande et l'afficher.

```
Algorithme CalculerFacture;
    Variable nbTicketsPetitDejeuner, nbTicketsDejeuner, nbTicketsDiner,
            montantCommande, montantVerse, montantManquant, monnaieRendue : réel;
    Début
        Ecrire("Entrez le nombre de tickets de petit déjeuner : ");
        Lire(nbTicketsPetitDejeuner);
        Ecrire("Entrez le nombre de tickets de déjeuner : ");
        Lire(nbTicketsDejeuner);
        Ecrire("Entrez le nombre de tickets de dîner : ");
        Lire(nbTicketsDiner);
        Ecrire("Entrez le montant versé par l'étudiant : ");
        Lire(montantVerse);

        montantCommande <- (nbTicketsPetitDejeuner * 75) +
                           (nbTicketsDejeuner + nbTicketsDiner) * 150;
        Ecrire("Le montant de la commande est de ", montantCommande:0:2);

        Si montantVerse >= montantCommande Alors
            Debut
                monnaieRendue <- montantVerse - montantCommande;
                Ecrire("La monnaie à rendre est de ", monnaieRendue:0:2);
            Fin
        Sinon
            Debut
                montantManquant <- montantCommande - montantVerse;
                Ecrire("Le montant manquant pour valider la commande ");
              Ecrire("est de ", montantManquant:0:2);
            Fin
        FinSi
    Fin.
```

Exercice 15 :

Proposer une "amélioration" de l'algorithme (algorigramme ou pseudo-code) précédent qui propose à l'étudiant une deuxième et dernière tentative lorsque la vérification ne s'est pas bien passée.

Analyse :
* Données d'entrée : le nombre de tickets de petit déjeuner, le nombre de tickets de déjeuner,
le nombre de tickets de dîner, le montant versé par l'étudiant.
* Données de sortie : le montant manquant pour valider la commande, la monnaie à rendre.
* Données intermédiaires : le montant de la commande.
* Méthode :
1. Lire le nombre de tickets de petit déjeuner.
2. Lire le nombre de tickets de déjeuner.
3. Lire le nombre de tickets de dîner.
4. Lire le montant versé par l'étudiant.
5. Calculer le montant de la commande.
6. Si le montant versé par l'étudiant est supérieur au montant de la commande, calculer la monnaie à rendre et l'afficher.
7. Sinon calculer le montant manquant pour valider la commande, l'afficher et relire le montant versé par l'étudiant.
8. Si le montant versé par l'étudiant est supérieur au montant de la commande, calculer la monnaie à rendre et l'afficher.
9. Sinon, afficher "Le montant versé par l'étudiant est insuffisant".

```
Algorithme CalculerFactureAmelioration;
    Variable nbTicketsPetitDejeuner, nbTicketsDejeuner, nbTicketsDiner,
            montantCommande, montantVerse, montantManquant, monnaieRendue : réel;
    Début
        Ecrire("Entrez le nombre de tickets de petit déjeuner : ");
        Lire(nbTicketsPetitDejeuner);
        Ecrire("Entrez le nombre de tickets de déjeuner : ");
        Lire(nbTicketsDejeuner);
        Ecrire("Entrez le nombre de tickets de dîner : ");
        Lire(nbTicketsDiner);
        Ecrire("Entrez le montant versé par l'étudiant : ");
        Lire(montantVerse);

        montantCommande <- (nbTicketsPetitDejeuner * 75) +
               (nbTicketsDejeuner + nbTicketsDiner) * 150;
        Ecrire("Le montant de la commande est de ", montantCommande:0:2);

        Si montantVerse >= montantCommande Alors
            Debut
                monnaieRendue <- montantVerse - montantCommande;
                Ecrire("La monnaie à rendre est de ", monnaieRendue:0:2);
            Fin
        Sinon
            Debut
                montantManquant <- montantCommande - montantVerse;
                Ecrire("Le montant manquant pour valider la commande ");
                 Ecrire("est de ", montantManquant:0:2);
                Ecrire("Entrez une nouvelle fois le montant versé par l'étudiant : ");
                Lire(montantVerse);
                Si montantVerse >= montantCommande Alors
                    Debut
                        monnaieRendue <- montantVerse - montantCommande;
                        Ecrire("La monnaie à rendre est de ", monnaieRendue:0:2);
                    Fin
                Sinon
                    Ecrire("Le montant versé par l'étudiant est insuffisant");
                FinSi
            Fin
        FinSi
    Fin.
```